

Project Documentation

Documentation

Date	02 October 2026
Student Name	Dhanyaa V S
Project Name	Automated Network Request Management in ServiceNow

1. Introduction & Project Overview

In traditional enterprise IT setups, handling network requests—such as setting up a new line, changing bandwidth, or updating connection settings—often relies on manual emails, paper slips, or scattered chat messages. This manual way of working causes unnecessary delays, missing details, untracked manager approvals, and potential security oversights.

To solve these operational problems, I developed the ‘Automated Network Request Management’ system on the ServiceNow cloud platform. As an individual developer, I configured and connected the entire end-to-end lifecycle: creating a dedicated Service Portal intake form, setting up dynamic Catalog UI Policies to keep data accurate, persisting data in a custom table Database Tables (u_database_tables) with the prefix ‘NETDB’, automating approvals and status updates using a 10-step Flow Designer workflow, and triggering automated email notifications. This implementation manual documents the exact steps I followed to build, configure, and verify this automated solution.

2. Stage 1: Custom Relational Schema Configuration (Database Tables)

To isolate network operational records from standard incident queues and support structured data retention, I created a custom table named ‘Database Tables’ (u_database_table) with dedicated business columns and auto-numbering configured with the prefix ‘NETDB’

Steps I followed:

1. I navigated to All > System Definition > Tables and clicked New.

2. I set the Table Label to 'Database Tables', and ServiceNow auto-generated the system name 'u_database_table'.
3. Under the Controls tab, I checked 'Auto-number', specified the Prefix as 'NETDB', and configured 7 digits so requests generate identifiers in the format NETDB0001001.
4. Under the Columns section, I defined the custom schema fields outlined in Table 1 below.
5. I saved the table configuration and validated the generated default Access Control Lists (ACLs).

Screenshot 1: Custom Table Dictionary Schema (sys_db_object)

The screenshot shows the ServiceNow configuration page for a table named 'Database Tables' (sys_db_object). The 'Columns' tab is active, displaying a list of 16 dictionary entries. The table configuration includes a label 'Database Tables', a name 'u_database_tables', and an application 'Global'. The columns list includes fields like 'Approval Status', 'Assignment to', 'Created', 'Created by', 'Customer Address', 'Database Number', 'Mobile Number', 'Mode of Payment', 'Number', 'Requested For', 'Sys ID', 'Total Amount', 'Type of Connection', 'Updated', 'Updated by', and 'Updates'.

Column label	Type	Reference	Max length	Default value	Display
Approval Status	String	(empty)	40		false
Assignment to	Reference	User	32		false
Created	Date/Time	(empty)	40		false
Created by	String	(empty)	40		false
Customer Address	String	(empty)	40		false
Database Number	String	(empty)	40		false
Mobile Number	String	(empty)	40		false
Mode of Payment	String	(empty)	40		false
Number	String	(empty)	40	javascript:global.getNextObjNumberPadded();	false
Requested For	String	(empty)	40		false
Sys ID	Sys ID (GUID)	(empty)	32		false
Total Amount	String	(empty)	40		false
Type of Connection	String	(empty)	40		false
Updated	Date/Time	(empty)	40		false
Updated by	String	(empty)	40		false
Updates	Integer	(empty)	40		false

Columns	Controls	Application Access
Extensible ☐ Live feed ☐		
Use auto-numbering to define a sequential identifying code made up of a prefix, a base number and a padding value to ensure a consistent format		
Prefix NETDB		
Number 1,000		
Number of digits 7		
Security Rules (ACLs) are required if anyone other than an administrator needs to work with this table. Creating default security rules will grant full access to this table to anyone with the user role you specify.		
Create access controls ☒		
* User role u_database_tables_user Q O		

Purpose: Validates the custom database schema structure, column types, and auto-number setup.

What I Captured: I opened System Definition > Tables > 'Database Tables' (u_database_tables) and captured the Columns list displaying my custom fields alongside the Auto-Number prefix 'NETDB'.

3. Stage 2: Service Catalog Intake & Dynamic UI Policy Configuration

Next, I configured the self-service catalog item within the Service Catalog and implemented dynamic client- side visibility logic to keep the user interface clean and prevent incomplete submissions.

Steps I followed:

1. I navigated to Service Catalog > Catalog Definitions > Maintain Items and created a new Catalog Item.
2. I set the Name to 'Network Request' and assigned it to the category 'Network request Management'.
3. In the Variables related list, I created all the intake variables corresponding to my database columns.
4. To handle conditional visibility, I opened the Catalog UI Policies related list and clicked New.
5. I configured the UI Policy condition as: 'Type of Connection is Existing', setting 'On Load' to True and 'Reverse if false' to True.
6. In the Catalog UI Policy Actions related list, I created an action targeting the 'Existing ID' variable with Make Visible = True and Make Mandatory = True.

Screenshot 2: Service Catalog Item Backend Configuration

<

≡

Catalog Item

Network Request

Press Alt+0 for help

Add relevant tags to the Meta field using comma-separated list of tags. These tags will be used while searching the item. Applicable only for keyword search when AI Search is enabled.

Meta

Copy

Try It

Update

Edit in Catalog Builder

Delete

Related Links

[Item Diagnostic](#)

[Show VA render type](#)

[Run Point Scan](#)

[\[SN Utils\] Versions \(25\)](#)

Variables (7)

Variable Sets (1)

Catalog UI Policies (2)

Catalog Client Scripts (1)

Available For

Not Available For

Categories (1)

Catalogs (1)

Catalog Data Lookup Definitions

Related Articles

Related Catalog Items

Assigned Topics

≡

▽

Order

Search

Actions on selected rows...

New

Catalog Item = Network Request

☐	Q	Type	Question	Order
		Reference	Requested For	100
		Single Line Text	Mobile Number	200
		Multiple Choice	Type of Connection	300
		Single Line Text	Enter your Existing ID	400
		Single Line Text	Total Amount	500
		Multiple Choice	Mode of Payment	600
		Single Line Text	Address	700

<<

<

1

to 7 of 7

>

>>

Purpose: Shows administrative catalog item configuration, variables list, and category alignment.

What I Captured: I navigated to Service Catalog > Maintain Items > ‘Network Request’ and captured the catalog item header, category selection, and the Variables related list displaying all the configured variables.

Screenshot 3: Dynamic Catalog UI Policy Configuration

Variables (7)

Variable Sets (1)

Catalog UI Policies (1)

Catalog Client Scripts (1)

Available For

Not Available For

Categories (1)

Catalogs (1)

Catalog Data Lookup Definitions

Related Articles

Related Catalog Items

Assigned Topics

≡

▽

Order

Search

Actions on selected rows...

New

Catalog Item = Network Request

☐	Q	Short description	Variable set	Conditions	Reverse if false	On load	Inherit	Updated	Order
		Display Existing ID when Type of Connection is Existing	(empty)		true	true	false	2026-09-30 02:52:26	100

<<

<

1

to 1 of 1

>

>>

Catalog UI Policy
Display Existing ID when Type of Connection is Existing

* Catalog Item: Network Request Active ☒

* Short description: Display Existing ID when Type of Connection is Existing

When to Apply **Script**

Catalog UI policy actions are applied only if all the following conditions are met:

1. The catalog UI policy is **Active**
2. The items in the **Conditions** field evaluate to true
3. The field specified in the catalog UI policy is present on the specified catalog item

Catalog Conditions

type_of_connection is Existing

Applies on a Catalog Item view ☒

Applies on Catalog Tasks ☐

Applies on Requested Items ☐

Apply the catalog UI policy actions when the form is loaded or when the user changes values on the form

On load ☒

Reverse the effects of the catalog UI policy actions when the Conditions evaluate to false

Reverse if false ☒

Related Links

[Run Point Scan](#)

[SN Utils | Versions \(1\)](#)

Catalog UI Policy Actions Actions on selected rows...

UI policy = Display Existing ID when Type of Connection is Existing

☐	Search Name	Read only	Mandatory	Visible	Order
☐	enter_your_existing_id	Leave alone	Leave alone	True	100

Purpose: Proves declarative condition logic driving conditional field display.

What I Captured: I opened the Catalog UI Policy record for ‘Network Request’ and captured the condition builder (‘Type of Connection is Existing’) along with the Catalog UI Policy Action making ‘existing_id’ visible and mandatory.

4. Stage 3: Service Portal User Intake Walkthrough (/sp)

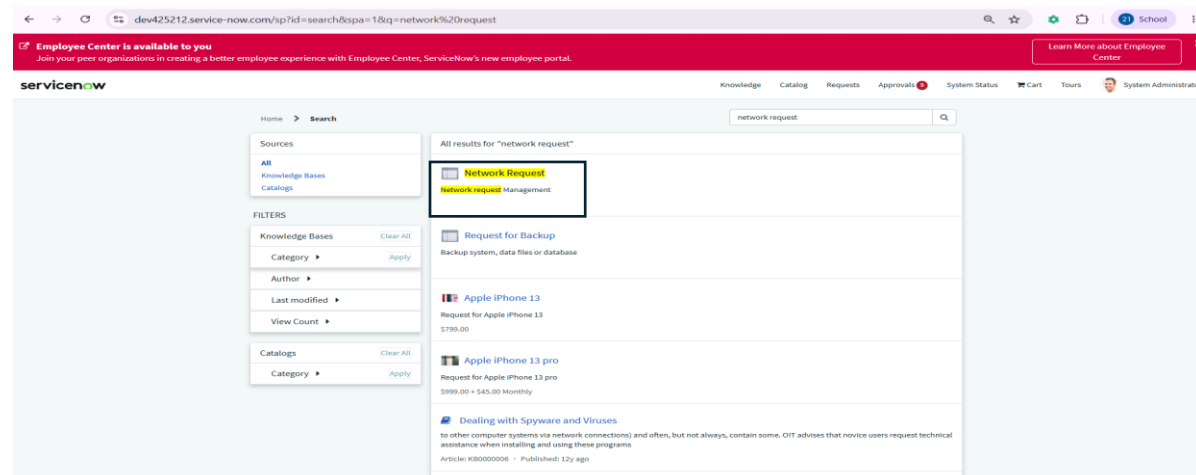
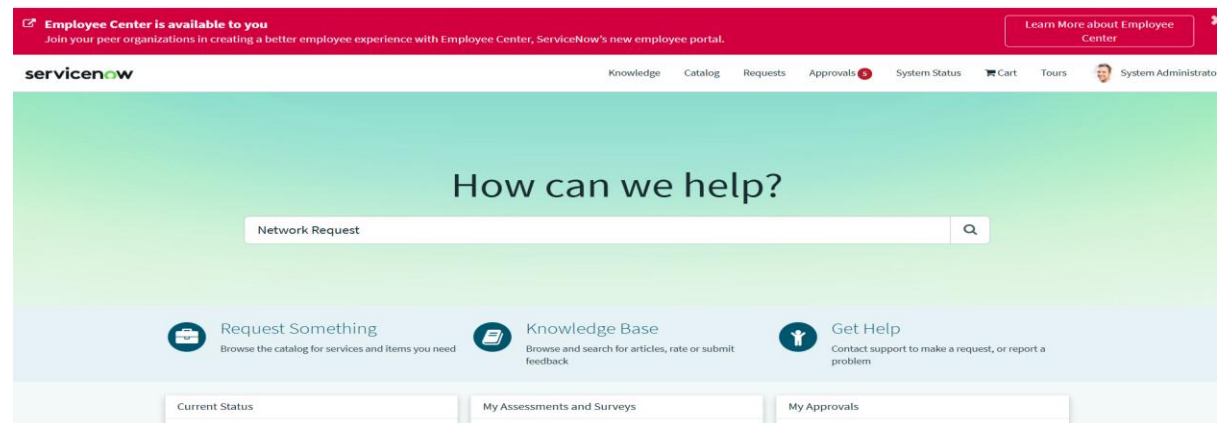
To test the requester interface, I simulated the end-user submission experience directly on the ServiceNow Service Portal.

Steps I followed:

1. I logged into the Service Portal via: <https://dev425212.service-now.com/sp>.
2. I entered ‘Network Request’ into the global search bar to confirm discoverability.
3. I opened the catalog item and reviewed the layout containing user contact, billing, and address fields.
4. I toggled the ‘Type of Connection radio button to ‘Existing’ to verify that the ‘Existing ID’ input field displayed dynamically.

5. I populated all mandatory fields, selected the Mode of Payment (UPI / Card), specified the Total Amount, entered the delivery Address, uploaded a test attachment, and clicked Submit.

Screenshot 4: Service Portal Catalog Search



Purpose: Shows end users discovering the Network Request catalog item within the self-service portal.

What I Captured: I navigated to /sp, typed 'Network Request' into the search box, and captured the catalog item result appearing under 'Network request Management'.

Screenshot 5: Network Request Catalog Form (Intake View)

Home > Service Catalog > Standard Changes > Network Standard Changes > Network Request

Search Catalog

Network Request

Network request Management

* Indicates required

* Requested For

System Administrator

Opened on behalf of

Email ID

User Name

Phone Number

* Mobile Number

* Type of Connection

☒ New

☐ Existing

Total Amount

Mode of Payment

☒ UPI

☐ CARD

* Address

Add attachments

Choose a file or drag it here.
Copy and paste clipboard files here.

Quantity: 1

Delivery Time: 0 Days

Add to Cart

Save as Draft

Order Now

Required Information

Mobile Number Address

Purpose: Displays the active catalog form capturing user contact, connection type, payment mode, and address.

What I Captured: I opened the form on /sp and captured the full intake screen with Requested For, contact info, Type of Connection (New/Existing), Total Amount, Mode of Payment (UPI/Card), Address, and the file attachment area.

Screenshot 6: Dynamic UI Policy in Action (Existing ID Conditional Field)

The screenshot displays a web application interface for a 'Network Request'. The breadcrumb navigation at the top reads: Home > Service Catalog > Standard Changes > Network Standard Changes > Network Request. A search bar labeled 'Search Catalog' is located to the right of the breadcrumbs.

The main form area is titled 'Network Request' with the subtitle 'Network request Management'. It contains several sections:

- * Indicates required**
- * Requested For**: A dropdown menu showing 'System Administrator'.
- Opened on behalf of**: A dropdown menu.
- Email ID**: A text input field.
- User Name**: A text input field.
- Phone Number**: A text input field.
- * Mobile Number**: A text input field.
- * Type of Connection**: Two radio buttons, 'New' and 'Existing'. The 'Existing' radio button is selected and highlighted with a green circle.
- Enter your Existing ID**: A text input field that appears only when the 'Existing' radio button is selected. This field is highlighted with a black rectangular box.
- Total Amount**: A text input field.
- Mode of Payment**: Two radio buttons, 'UPI' (selected) and 'CARD'.
- * Address**: A text input field.
- Add attachments**: A dashed box with a cloud icon and the text 'Choose a file or drag it here.'

On the right side of the form, there is a sidebar with the following elements:

- Quantity**: A dropdown menu showing '1'.
- Delivery Time: 0 Days**
- Add to Cart**: A button with a shopping cart icon.
- Save as Draft**: A button with a document icon.
- Order Now**: A large blue button.
- Required information**: A section with two red labels: 'Mobile Number' and 'Address'.

Purpose: Demonstrates client-side dynamic UI policy execution revealing the 'Existing ID' field when 'Existing' is selected.

What I Captured: On the portal form, I clicked the Existing radio choice and captured the screen showing the mandatory Existing ID field dynamically appearing without a page refresh.

Screenshot 7: Request Submission Summary (Order Confirmation)

Home > Service Catalog > Standard Changes > Network Standard Changes > Network Request

Search Catalog

Network Request

Network request Management

* Indicates required

* Requested For

Abel Tuter

Opened on behalf of

Abraham Lincoln

Email ID

abraham.lincoln@example.com

User Name

abraham.lincoln

Phone Number

(555) 555-0004

* Mobile Number

9518476203

* Type of Connection

☒ New

☐ Existing

Total Amount

\$100

Mode of Payment

☒ UPI

☐ CARD

* Address

123 Main Street, Suite 400, Springfield, IL 62701, USA

Add attachments

Quantity: 1

Delivery Time: 0 Days

Add to Cart

Save as Draft

Order Now

Home > Request Summary - REQ0010012

Search Catalog

Submitted : 2026-10-07 00:00:45

Request Number : **REQ0010012**

Requested for : Abel Tuter

Estimated Delivery : 2026-10-07

Item	Delivery Date	Stage	Price (each)	Quantity	Total
Network Request	2026-10-07		---	1	---
					Total: \$0.00

Purpose: Shows the portal order confirmation screen confirming successful request generation.
What I Captured: I clicked Submit and captured the resulting confirmation screen displaying the generated Request Number (REQ... / RITM...) and timestamp.

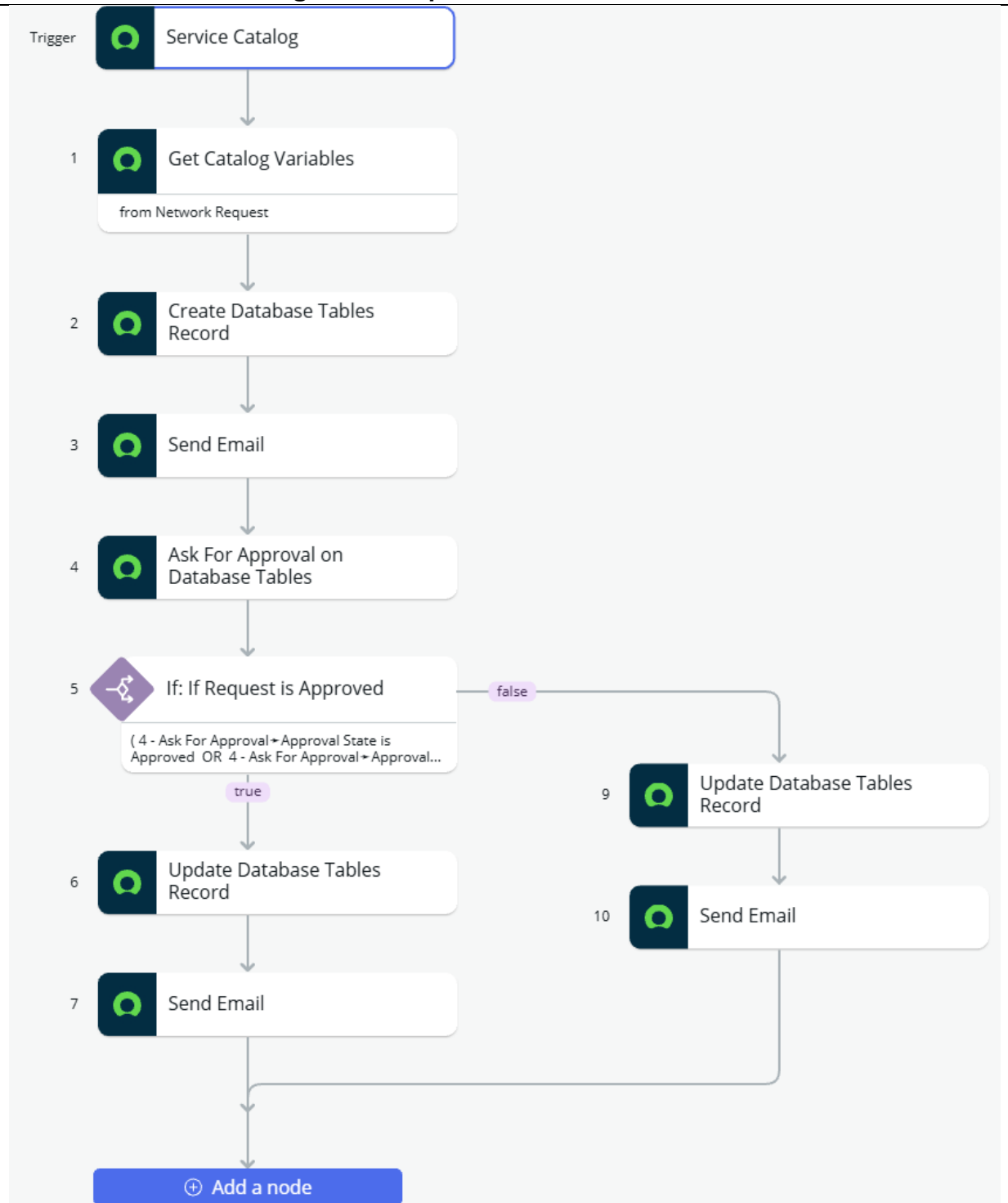
Stage 4: Flow Designer 10-Step Workflow Construction

To automate the entire request lifecycle without writing custom scripts, I built a zero-code workflow in ServiceNow Flow Designer / Workflow Studio.

Steps I followed to configure the 10-step flow:

- **Trigger:** I configured the trigger as Service Catalog so the flow fires automatically upon form submission.
- **Step 1 (Get Catalog Variables):** I added the Get Catalog Variables action, pointed it to the 'Network Request' catalog item, and pulled all form variables into data pills.
- **Step 2 (Create Record):** I added the Create Record action targeting Database Tables (u_database_tables). I mapped all incoming catalog data pills to the table fields and set State = Awaiting Approval.
- **Step 3 (Send Email):** I added the Send Email action targeting the created Database Tables record to send an instant acknowledgment email to the requester containing the NETDB tracking ID.
- **Step 4 (Ask For Approval):** I added the Ask For Approval action on the Database Tables record, setting the rule to approve when Anyone approves and targeting the designated manager.
- **Step 5 (If Condition):** I configured an If condition evaluating: Step 4 Ask For Approval > Approval State is Approved.
- **True Branch (Steps 6 & 7):** Under the True condition, I added Step 6 Update Record to change State to Approved, followed by Step 7 Send Email to deliver an approval notification.
- **False Branch (Steps 9 & 10):** Under the Else condition, I added Step 9 Update Record to change State to Rejected, followed by Step 10 Send Email to deliver a rejection notification.
- I clicked Save, executed a test run using Test Flow, and activated the workflow.

Screenshot 8: Flow Designer 10-Step Workflow Execution Canvas



Purpose: Proves end-to-end low-code workflow logic in ServiceNow Workflow Studio.

What I Captured: I opened Flow Designer for Network Request and captured the complete workflow canvas showing the Trigger, Steps 1 to 4, the Step 5 If logic, and both execution branch paths.

Screenshot 9: Flow Execution Details (Runtime Traces & Logs)

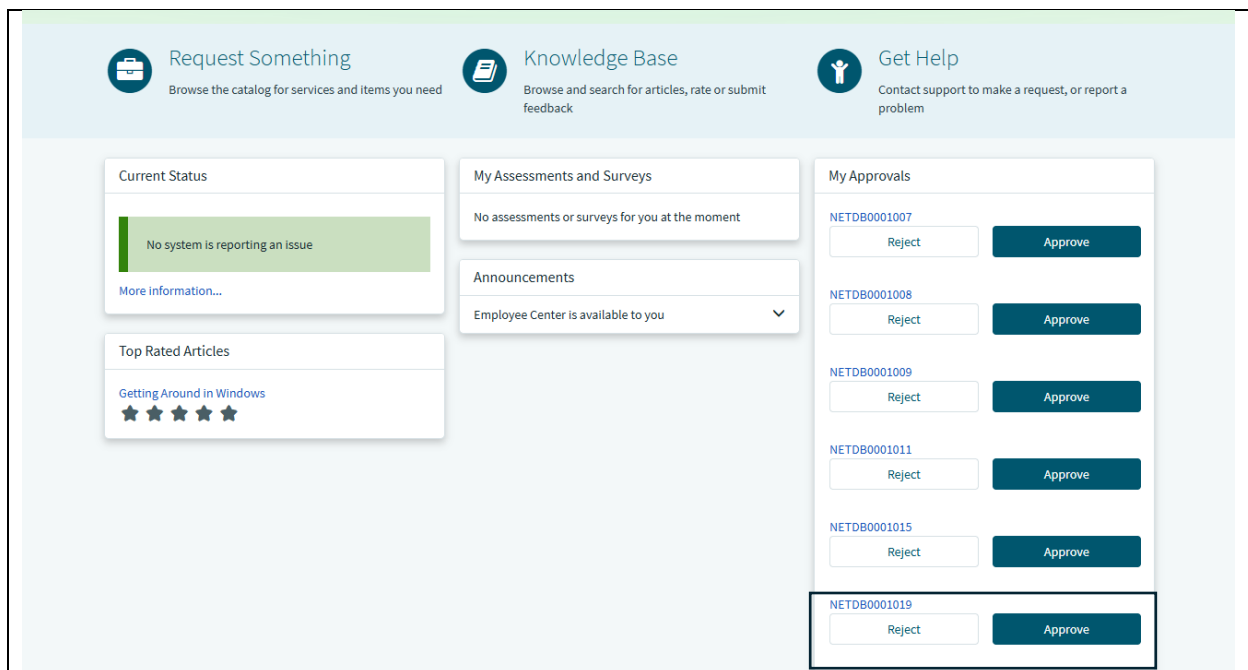
EXECUTION DETAILS

Network Request

Test Run - Waiting ⚠️
Cancel flow
Open flow
Open context record

Show Action Details
State
Start time

FLOW STATISTICS		Run as System Administrator	Open flow logs	Waiting	2026-10-07 01:26:20	3030ms
TRIGGER						
Catalog Item Requested						
ACTIONS						
1	Get Catalog Variables from Network Request	Core Action	Completed	2026-10-07 01:26:20	632ms	
2	Create Record	Core Action	Completed	2026-10-07 01:26:21	190ms	
3	Send Email		Completed	2026-10-07 01:26:21	694ms	
4	Ask For Approval	Core Action	Waiting	2026-10-07 01:26:22	1223ms	
▼ 5	If If Request Is Approved	Flow Logic	Not Run			
6	Update Record	Core Action	Not Run			
7	Send Email		Not Run			
▼ 8	● Else	Flow Logic	Not Run			
9	Update Record	Core Action	Not Run			
10	Send Email		Not Run			
ERROR HANDLER						



Purpose: Confirms runtime execution success and data pill variable parsing without errors.

What I Captured: I opened the Flow Execution Details for my test submission and captured the execution trace showing green checkmarks across all evaluated steps.

Screenshot 10: Flow Execution Details (Post-Approval Resumption - State: Completed / True Branch Executed)

FLOW STATISTICS			
TRIGGER		Run as: System Administrator	Open flow logs 🔗
 Catalog Item Requested		Completed	2026-10-07 01:26:20
ACTIONS			
1	 Get Catalog Variables from Network Request	Core Action	Completed
2	 Create Record	Core Action	Completed
3	 Send Email		Completed
4	 Ask For Approval	Core Action	Completed
▼ 5	 If Request Is Approved	Flow Logic	Evaluated - True
6	 Update Record	Core Action	Completed
7	 Send Email		Completed
▼ 8	 Else	Flow Logic	Not Run
9	 Update Record	Core Action	Not Run
10	 Send Email		Not Run
ERROR HANDLER			

Purpose: Demonstrates that upon the manager clicking 'Approve', the flow automatically resumes, evaluates the Step 5 condition as True, and executes Steps 6 and 7 to completion.

What I Captured: In Flow Execution Details after approving the request, I refreshed the execution view and captured the updated trace showing overall Flow State = Completed, Step 4 completed (green checkmark), Step 5 evaluated as True, and Steps 6 (Update Record to Approved) and 7 (Send Approval Email) successfully completed with green checkmarks.

6. Stage 5: Data Persistence, Governance & Notification Verification

Finally, I verified that submitted requests were saved to the database, linked to approval records, and sent milestone notifications.

Steps I followed:

1. I navigated to Database Tables (`u_database_table.list`) and verified that a record with the prefix NETDB (e.g., `NETDB0001001`) was generated with State = Awaiting Approval.
2. I opened `sysapproval_approver_list.do` to verify that an approval task linked to the NETDB record was assigned to the approving manager.
3. I simulated the approval action, which triggered the flow to update the table state to Approved.
4. I navigated to System Logs > Emails (`sys_email.list`) to verify that outbound SMTP messages were generated and queued for both the initial acknowledgment and approval confirmation.
5. I verified table-level Access Control Lists (ACLs) to ensure that regular requesters only have read access to their own tickets, while administrators maintain write access.

Screenshot 10: Database Tables Record View (u_database_tables)

Database Tables										Number	Search	Actions on selected rows...			New
All															
☐	☐	Number	Approval Status	Assignment to	Customer Address	Database Number	Mobile Number	Mode of Payment	Requested For	Total Amount	Type of				
		NETDB0001014	Requested	(empty)		RITM0000001									
		NETDB0001010	Approved	ITIL User		RITM0010009		UPI			New				
		NETDB0001005	Approved	ITIL User	123 street	RITM0010007	9876543210	UPI	Abel Tuter	\$50	New				
		NETDB0001008	Requested	(empty)	123 street	RITM0010008	987452310	UPI			New				
		NETDB0001017	Approved	ITIL User	123 street	RITM0010011	9518476230	UPI	62826bf03710200044e0bfc8bcbe5df1	\$50	New				
		NETDB0001011	Requested	(empty)	123 street	RITM0010010	9518476230	UPI	5408091a3b100300e81d47b334efc452	\$50	New				
		NETDB0001019	Requested	(empty)	123 Main Street, Suite 400, Springfield,	RITM0010012	9518476203	UPI	62826bf03710200044e0bfc8bcbe5df1	\$100	New				
		NETDB0001016	Approved	ITIL User	123 street	RITM0010011	9518476230	UPI	62826bf03710200044e0bfc8bcbe5df1	\$50	New				
		NETDB0001007	Requested	(empty)	123 street	RITM0010008	987452310	UPI			New				
		NETDB0001015	Requested	(empty)	123 street	RITM0010011	9518476230	UPI	62826bf03710200044e0bfc8bcbe5df1	\$50	New				
		NETDB0001004	Requested	(empty)		RITM0010006		UPI			New				
		NETDB0001012	Approved	ITIL User	123 street	RITM0010010	9518476230	UPI	5408091a3b100300e81d47b334efc452	\$50	New				
		NETDB0001013	Requested	(empty)		RITM0010001									
		NETDB0001009	Requested	(empty)		RITM0010009		UPI			New				
		NETDB0001006	Approved	ITIL User	123 street	RITM0010007	9876543210	UPI	Abel Tuter	\$50	New				

Purpose: Validates custom persistence and auto-numbering generation in the backend table.

What I Captured: I opened the generated record in u_database_table.list and captured the form displaying the auto-numbered NETDB key (e.g., NETDB0001001), State ('Awaiting Approval'), contact fields, and payment parameters.

Screenshot 11: Approval List & Approver View (sysapproval_approver)

State	Approver	Comments	Approval for	Created	Approving
Approved	System Administrator		(empty)	2026-10-02 00:18:15	Database Tables: NETDB0001006
Approved	System Administrator		(empty)	2026-10-02 03:03:48	Database Tables: NETDB0001010
Requested	System Administrator		(empty)	2026-10-02 03:02:07	Database Tables: NETDB0001009
Approved	System Administrator		(empty)	2026-10-07 01:26:22	Database Tables: NETDB0001019
Requested	System Administrator		(empty)	2026-10-02 02:35:55	Database Tables: NETDB0001008
Approved	System Administrator		(empty)	2026-10-02 00:16:03	Database Tables: NETDB0001005
Requested	System Administrator		(empty)	2026-10-03 23:17:52	Database Tables: NETDB0001015
Approved	System Administrator		(empty)	2026-10-03 23:18:26	Database Tables: NETDB0001016
Requested	System Administrator		(empty)	2026-10-02 02:35:15	Database Tables: NETDB0001007
Approved	System Administrator		(empty)	2026-10-02 08:23:59	Database Tables: NETDB0001012
Approved	System Administrator		(empty)	2026-10-03 23:27:36	Database Tables: NETDB0001017
Requested	System Administrator		(empty)	2026-10-02 08:22:42	Database Tables: NETDB0001011
Approved	System Administrator		RITM0010006	2026-10-01 01:58:25	(empty)

Purpose: Proves governance integration linking the custom table to the platform approval engine.

What I Captured: I opened sysapproval_approver_list.do and captured the approval record showing Approval for = NETDB0001001 , Approver = System Administrator, and State = Requested (or Approved).

Screenshot 12: Outbound Email Notification Verification (sys_email)

Emails

Subject

Search

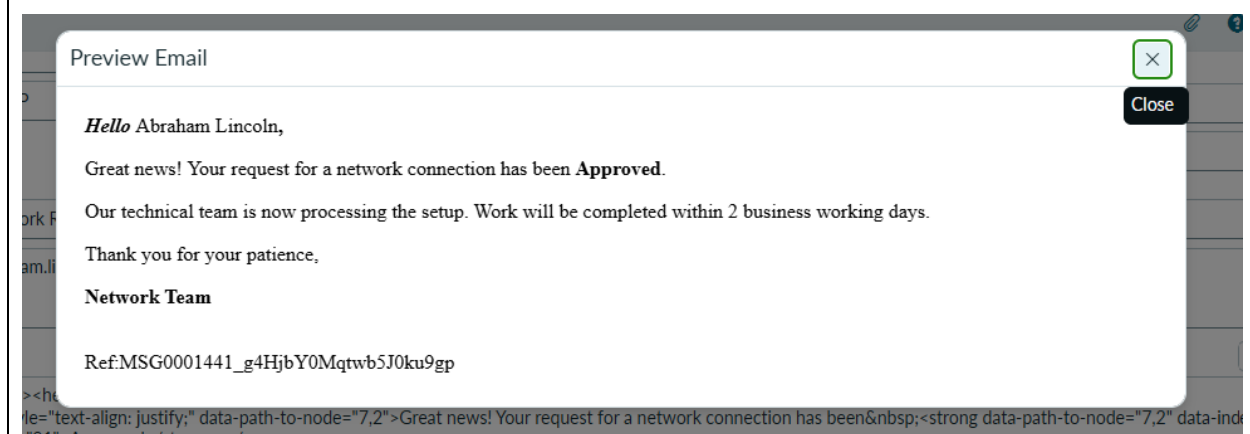
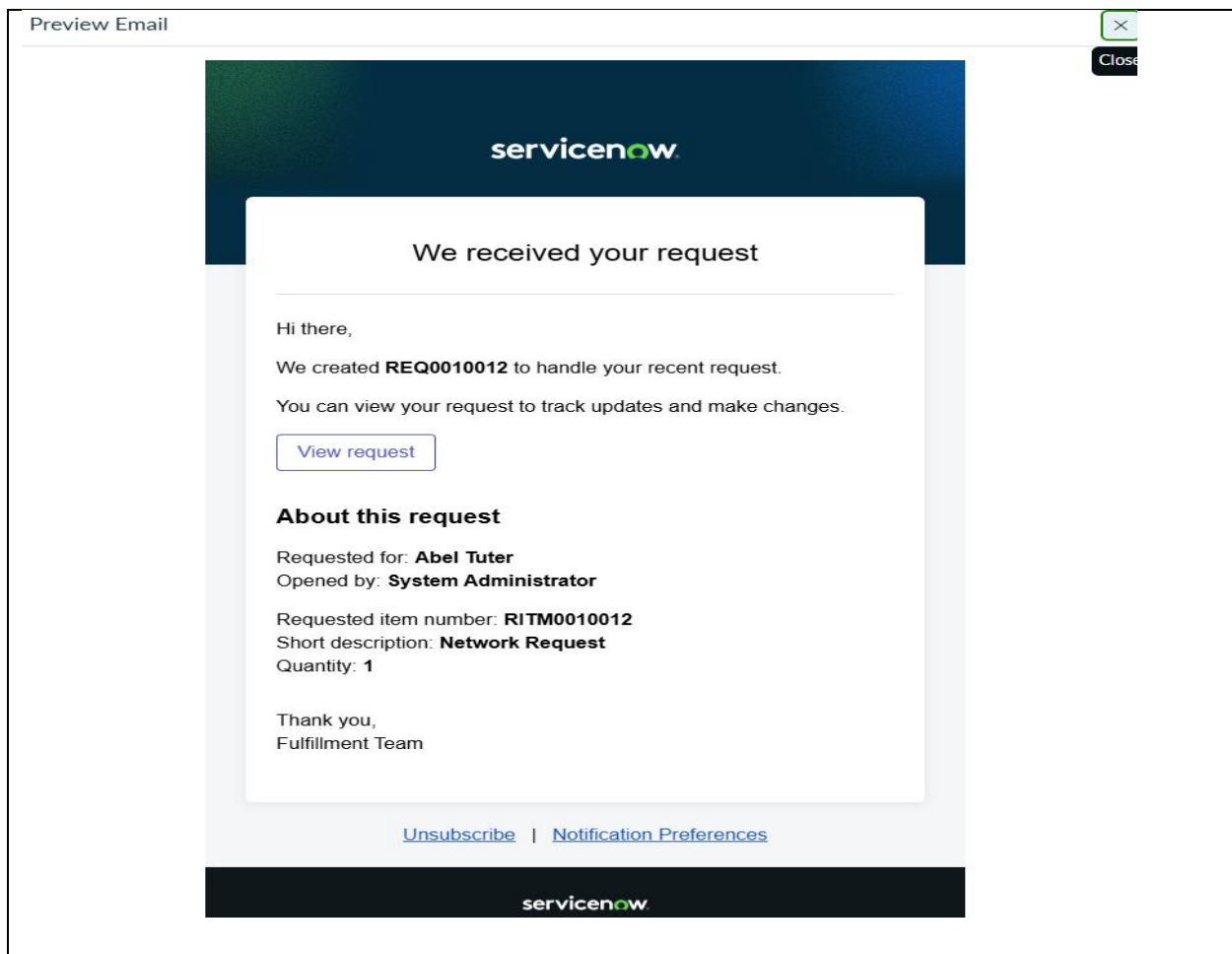
Actions on selected rows...

New

All > Created on Today

	Created	Recipients	Subject	Type	Notification type	User ID
	2026-10-07 00:00:19	admin@example.com	Daily job to fetch Email Indicator Data and Email Notifications created completed with error	send-ready	SMTP	(empty)
	2026-10-07 01:26:21	abraham.lincoln@example.com	Database Record is Created	send-ready	SMTP	(empty)
	2026-10-07 00:01:38	abel.tuter@example.com.admin@example.com	Request REQ0010012 was approved	send-ready	SMTP	(empty)
	2026-10-07 00:00:53	abel.tuter@example.com.admin@example.com	Request REQ0010012 was created	send-ready	SMTP	(empty)
	2026-10-07 01:00:06	alileen.motern@example.com	Restocking Request For APC 42U 3100 SP2 NetShelter Rack	send-ready	SMTP	(empty)
	2026-10-07 01:00:12	alileen.motern@example.com	Restocking Request For Dell Inc. PowerEdge M710HD Blade Server	send-ready	SMTP	(empty)
	2026-10-07 01:00:09	alileen.motern@example.com	Restocking Request For Fujitsu 1TB Hybrid Solid State Drive	send-ready	SMTP	(empty)

Emails		Subject	Search				Actions on selected rows...		New
All > Created on Today									
	Q	Created	Recipients	Subject	Type	Notification type	User ID		
		2026-10-07 00:00:19	admin@example.com	Daily job to fetch Email Indicator Data and Email Notifications created completed with error	send-ready	SMTP	(empty)		
		2026-10-07 01:26:21	abraham.lincoln@example.com	Database Record is Created	send-ready	SMTP	(empty)		
		2026-10-07 01:50:15	abraham.lincoln@example.com	Network Request Approved -	send-ready	SMTP	(empty)		
		2026-10-07 00:01:38	abel.tuter@example.com,admin@example.com	Request REQ0010012 was approved	send-ready	SMTP	(empty)		
		2026-10-07 00:00:53	abel.tuter@example.com,admin@example.com	Request REQ0010012 was created	send-ready	SMTP	(empty)		
		2026-10-07 01:00:06	alileen.mottern@example.com	Restocking Request For APC 42U 3100 SP2 NetShelter Rack	send-ready	SMTP	(empty)		
		2026-10-07 01:00:12	alileen.mottern@example.com	Restocking Request For Dell Inc. PowerEdge M710HD Blade Server	send-ready	SMTP	(empty)		
		2026-10-07 01:00:09	alileen.mottern@example.com	Restocking Request For Fujitsu 1TB Hybrid Solid State Drive	send-ready	SMTP	(empty)		



Purpose: Validates automated communications dispatching receipt confirmation to the user.

What I Captured: I opened System Logs > Emails (sys_email.list), located the sent message, and captured the email record showing recipient address, subject ('Network Request Received - Ref: NETDB...'), and body content.

Screenshot 13: Access Control Lists (ACL) Security Configuration

Access Controls (4)

Security Data Filters

Labels (1)

Database Indexes (2)

Table Subscription Configuration (1)

≡

🔍

Name

Search

⚙️

📄

🔍

📄

—

Actions on selected rows...

Access Controls

☐

🔍

Name ▲

Decision Type

⋮

Operation

Type

Active

Updated by

Updated

u_database_tables

Allow If

write

record

true

admin

2026-09-29 23:17:45

u_database_tables

Allow If

read

record

true

admin

2026-09-29 23:17:43

u_database_tables

Allow If

create

record

true

admin

2026-09-29 23:17:40

u_database_tables

Allow If

delete

record

true

admin

2026-09-29 23:17:48

Purpose: Proves role-based access control and security rules enforced on the custom table.

What I Captured: I navigated to the table's Access Controls related list and captured the active ACL rules (u_database_table create, read, write) showing enforced role permissions

7. Conclusion & Project Retrospective

By following this structured, step-by-step implementation, I successfully automated the entire network request management process within ServiceNow. Creating the custom table Database Tables (u_database_table) gave me complete control over the data schema and auto-numbering using the prefix NETDB, while the Catalog UI Policies guaranteed that requesters only fill in necessary, relevant details.

Using Flow Designer allowed me to establish an enterprise-grade approval and notification workflow without writing brittle custom scripts. Requesters now benefit from an intuitive self-service portal with instant email updates, managers have transparent approval controls, and all operations are securely logged for auditability. Overall, this project shows how native low-code tools in ServiceNow can transform slow, manual IT processes into fast, reliable, and auditable automated workflows.